

Microscopy Data Management API

Role: Backend Developer

Difficulty: junior

Project Type: backend

Duration: 3 days

Focus: fullstack

Candidate: Nikita Rane

Project Overview

Create a REST API to store and retrieve microscopy image metadata, with a simple React UI for upload and list.

Technologies

- Express.js
- Node.js
- React
- Mongoose
- MongoDB

Requirements

- Build a practical, measurable project that matches the student's current skills and education. Use the student's existing skills where appropriate while introducing a reasonable amount of new learning. The project should be suitable for evaluating implementation and problem-solving ability.

Objectives

- Design MongoDB schema for image metadata
- Implement CRUD endpoints
- Build React component to upload images
- Validate input
- Write unit tests

Functional Requirements

- POST /images to upload metadata
- GET /images to list
- GET /images/:id to get details
- PUT /images/:id to update
- DELETE /images/:id to delete
- Store image files locally

Non-Functional Requirements

- Use environment variables
- Handle errors gracefully
- Validate file type
- Limit upload size to 5MB
- Use HTTPS

Modules

server

Express app entry point

- app.js
- server.js

routes

Image routes

- imageRoutes.js

controllers

Image controller logic

- imageController.js

models

Mongoose schema

- Image.js

middleware

File upload handling

- upload.js

utils

Helper functions

- logger.js

Implementation Plan

Setup

- Initialize npm, install dependencies
- Create .env template

Schema

- Define Image schema
- Add indexes

Routes

- Create CRUD routes
- Attach middleware

Upload

- Configure multer
- Validate file size/type

Frontend

- Create React upload form
- Display image list

Test

- Write unit tests
- Run integration tests

Evaluation Criteria

API correctness (30%)

All endpoints work as specified

Code quality (20%)

Clean, modular code

Error handling (15%)

Graceful error responses

Security (10%)

Basic security measures

Documentation (10%)

Clear README and API docs

Test coverage (15%)

Adequate unit/integration tests

Deliverables

- Git repo
- API documentation
- React UI
- Test suite
- README
- Postman collection

Suggested Folder Structure

- server/

- client/
- server/models/
- server/routes/
- server/controllers/
- client/src/

API Endpoints

GET /images

List images

POST /images

Upload image metadata

GET /images/:id

Get image details

PUT /images/:id

Update image metadata

DELETE /images/:id

Delete image

Database Design

images

Store image metadata

- _id
- title
- description
- filename
- uploadDate
- tags
- size

Testing Plan

- Unit tests for controllers
- Integration tests for routes
- Frontend component tests
- File upload size test
- Error handling test
- Database connection test

Candidate Fit

The project leverages Nikita's data recording skills by focusing on metadata management, while introducing backend fundamentals and file handling.